

1 Incarcare date

In aceasta sectiune definim caile catre setul de date NSL-KDD gazduit pe platforma Kaggle. Deoarece fisierele originale nu contin un antet (header) cu numele coloanelor, definim manual lista acestora inainte de a incarca datele intr-un format tabelar folosind biblioteca Pandas.

```
1 import os
2 import pandas as pd
3
4 # Pe Kaggle - calea catre NSL-KDD
5 DATA_PATH = '/kaggle/input/nsldata'
6
7 # Listeaza fisierele disponibile
8 for root, dirs, files in os.walk(DATA_PATH):
9     for f in files:
10         fpath = os.path.join(root, f)
11         size_mb = os.path.getsize(fpath) / 1e6
12         print(f'{fpath} ({size_mb:.1f} MB)')
13
14 # Definim manual coloanele deoarece fisierul NSL-KDD nu are
15 # header
16 kdd_cols = ['duration', 'protocol_type', 'service', 'flag',
17             'src_bytes', 'dst_bytes', 'land',
18             'wrong_fragment', 'urgent', 'hot', '
19             num_failed_logins', 'logged_in', 'num_compromised',
20             'root_shell', 'su_attempted', 'num_root', '
21             num_file_creations', 'num_shells',
22             'num_access_files', 'num_outbound_cmds', '
23             is_host_login', 'is_guest_login', 'count',
24             'srv_count', 'serror_rate', 'srv_serror_rate', '
25             rerror_rate', 'srv_rerror_rate',
26             'same_srv_rate', 'diff_srv_rate', '
27             srv_diff_host_rate', 'dst_host_count', '
28             dst_host_srv_count',
29             'dst_host_same_srv_rate', '
30             dst_host_diff_srv_rate', 'dst_host_same_src_port_rate',
31             'dst_host_srv_diff_host_rate', '
32             dst_host_serror_rate', 'dst_host_srv_serror_rate',
33             'dst_host_rerror_rate', '
34             dst_host_srv_rerror_rate', 'label', 'difficulty']
35
36 # Cautam automat fisierul de train ignorand numele
37 # folderelor
38 train_file = None
39 for root, dirs, files in os.walk('/kaggle/input'):
40     for f in files:
41         if 'kddtrain' in f.lower() and f.endswith('.txt'):
42             train_file = os.path.join(root, f)
43             break
```

```

32     if train_file:
33         break
34
35 # Citim datele
36 if train_file is not None:
37     print(f"SUCCESE: Am gasit datele la calea -> {train_file}")
38     df = pd.read_csv(train_file, names=kdd_cols)
39
40     # Eliminam coloana 'difficulty' (nu ne ajuta la
    predictii)
41     if 'difficulty' in df.columns:
42         df.drop('difficulty', axis=1, inplace=True)
43
44     print(f"Datele au fost incarcate cu succes! Linii: {df.
    shape[0]} | Coloane: {df.shape[1]}")
45     display(df.head())
46 else:
47     print("EROARE CRITICA: Kaggle inca proceseaza fisierele
    sau dataset-ul este gol.")

```

Datele au fost incarcate cu succes! Linii: 125973 | Coloane: 42

	duration	protocol_type	service	flag	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	...	dst_host_srv_count	dst_host_same_srv_rate	dst_host_diff_srv_rate	dst_host_same_src_i
0	0	tcp	ftp_data	SF	491	0	0	0	0	0	...	25	0.17	0.03	
1	0	udp	other	SF	146	0	0	0	0	0	...	1	0.00	0.60	
2	0	tcp	private	SF	0	0	0	0	0	0	...	26	0.10	0.05	
3	0	tcp	http	SF	232	8153	0	0	0	0	...	255	1.00	0.00	
4	0	tcp	http	SF	199	420	0	0	0	0	...	255	1.00	0.00	

Figura 1: Vizualizarea fişierelor detectate în directorul NSL-KDD şi a primelor randuri din dataset.

2 Datele utilizate

O etapă crucială în machine learning este înţelegerea datelor cu care lucrăm. Mai jos extragem coloana tinta (label), care indică dacă traficul este normal sau reprezintă un atac, şi examinăm tipurile de atacuri prezente.

```

1 # Identifica coloana label
2 label_col = 'label'
3 print(f'Coloana label: "{label_col}"')
4
5 # Distributia claselor
6 class_counts = df[label_col].value_counts()
7 print(f'\nClase gasite ({len(class_counts)}):')
8 print(class_counts)

```

```

Coloana label: "label"
Class gasite (23):
label
normal      67343
neptune     41214
satan       3633
ipsweep     3599
portsweep   2931
smurf       2646
rmap        1493
back        956
teardrop    892
warezclient 890
pod         201
guess_passwd 53
buffer_overflow 30
warezmaster 20
land        18
imap        11
rootkit     10
loadmodule  9
ftp_write    8
multihop    7
phf         4
perl        3
npy         2
Name: count, dtype: int64

```

Figura 2: Lista claselor identificate si numarul de inregistrari pentru fiecare.

2.1 Distributia Claselor

Setul de date NSL-KDD este recunoscut pentru dezechilibrul claselor (class imbalance). In codul de mai jos am generat grafice pentru a vizualiza distributia absoluta a celor mai frecvente 15 tipuri de trafic si distributia procentuala a top 5 clase.

```

1 import matplotlib.pyplot as plt
2
3 # Vizualizare distributie clase
4 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
5
6 # Absolut - doar top 15 atacuri pentru ca NSL-KDD are multe
   clase mici
7 class_counts.head(15).plot(kind='bar', ax=ax1, color='
   steelblue', edgecolor='black')
8 ax1.set_title('Distributia claselor - Top 15 (absolut)',
   fontsize=13)
9 ax1.set_xlabel('Clasa')
10 ax1.set_ylabel('Nr. inregistrari')
11 ax1.tick_params(axis='x', rotation=45)
12
13 # Procentual - doar top 5 pentru claritate
14 top_5 = class_counts.head(5)
15 top_5.plot(kind='pie', ax=ax2, autopct='%1.1f%%', startangle
   =90)
16 ax2.set_title('Distributia claselor - Top 5 (%)', fontsize
   =13)
17 ax2.set_ylabel('')
18
19 plt.tight_layout()
20 plt.show()
21
22 print(f'\nImbalance ratio (Top vs Bottom): {class_counts.max
   () / class_counts.min():.1f}x')

```

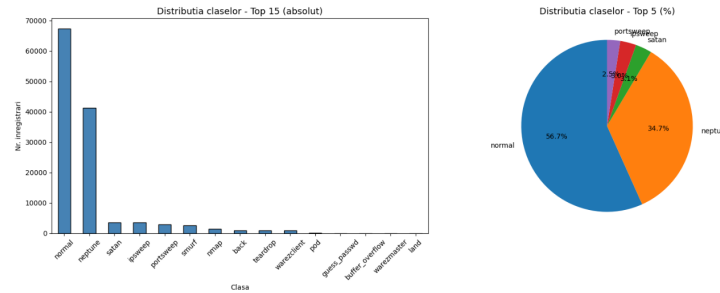


Figura 3: Grafice reprezentand dezechilibrul claselor in setul de date.

2.2 Numar de date de antrenare

Inainte de preprocesare, verificam statisticile generale ale setului de date: numarul total de attribute, valorile lipsa (NaN), valorile infinite si inregistrările duplicate care ar putea altera procesul de invatare.

```
1 import numpy as np
2
3 # Info general
4 print('== Statistici generale ==')
5 print(f'Randuri: {df.shape[0]:,}')
6 print(f'Coloane: {df.shape[1]:,}')
7 print(f'Valori lipsa: {df.isnull().sum().sum():,}')
8 print(f'Valori infinite: {np.isinf(df.select_dtypes(include=
9     np.number)).sum().sum():,}')
10 print(f'Duplicate: {df.duplicated().sum():,}')
11 print()
12 df.describe()
```

```
== Statistici generale ==
Randuri: 125,973
Coloane: 42
Valori lipsa: 0
Valori infinite: 0
Duplicate: 0

[6]:
```

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	dst_host_count	ds
count	125973.00000	1.259730e+05	1.259730e+05	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	125973.000000	
mean	287.14465	4.556674e+04	1.977911e+04	0.000198	0.022887	0.000111	0.204409	0.001222	0.395736	0.279250	182.148945	
std	2404.51531	5.870331e+06	4.021269e+06	0.014086	0.253530	0.014366	2.149968	0.045239	0.489010	23.942042	99.206213	
min	0.00000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	
25%	0.00000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	82.000000	
50%	0.00000	4.400000e+01	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	255.000000	
75%	0.00000	2.760000e+02	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	1.000000	0.000000	255.000000	
max	42908.00000	1.379964e+09	1.309937e+09	1.000000	3.000000	3.000000	77.000000	5.000000	1.000000	7479.000000	255.000000	

Figura 4: Statistici descriptive ale setului brut de date.

3 Preprocesare

Datele brute necesita curatare inainte de a fi trimise modelului. Procesul de mai jos include eliminarea valorilor infinite si duplicate, aruncarea coloanelor cu

varianta zero (care nu ofera nicio informatie utila), transformarea datelor de tip text in format numeric (Label Encoding) si inlaturarea claselor extrem de rare.

```
1 from sklearn.preprocessing import LabelEncoder
2
3 df_clean = df.copy()
4
5 # 1. Inlocuieste inf cu NaN, apoi elimina
6 df_clean.replace([np.inf, -np.inf], np.nan, inplace=True)
7 before = len(df_clean)
8 df_clean.dropna(inplace=True)
9 print(f'Randuri eliminate (NaN/inf): {before - len(df_clean)
    :}')
10
11 # 2. Elimina duplicate
12 before = len(df_clean)
13 df_clean.drop_duplicates(inplace=True)
14 print(f'Randuri eliminate (duplicate): {before - len(
    df_clean):}')
15
16 # 3. Elimina features cu varianta 0 (doar pentru numerice)
17 numeric_cols = df_clean.select_dtypes(include=np.number).
    columns.tolist()
18 zero_var_cols = [c for c in numeric_cols if df_clean[c].var
    () == 0]
19 df_clean.drop(columns=zero_var_cols, inplace=True)
20 print(f'Coloane eliminate (var=0): {len(zero_var_cols)} -> {
    zero_var_cols[:5]}...')
21
22 # 4. Encoding pentru categorice (Protocol, Service, Flag)
23 cat_cols = ['protocol_type', 'service', 'flag']
24 for c in cat_cols:
25     if c in df_clean.columns:
26         le_cat = LabelEncoder()
27         df_clean[c] = le_cat.fit_transform(df_clean[c].
            astype(str))
28 print('Coloane categorice encodeate cu succes.')
29
30 # 5. Eliminam clase rare (<10 samples) ca sa nu dea eroare
    train_test_split (stratify)
31 counts = df_clean[label_col].value_counts()
32 rare_classes = counts[counts < 10].index
33 if len(rare_classes) > 0:
34     df_clean = df_clean[~df_clean[label_col].isin(
        rare_classes)]
35     print(f'Eliminate {len(rare_classes)} clase rare (<10
        exemple)')
36
37 # 6. Encoding label
38 le = LabelEncoder()
```

```

39 df_clean['label_enc'] = le.fit_transform(df_clean[label_col
    ])
40 class_names = list(le.classes_)
41 num_classes = len(class_names)
42 print(f'\nClase finale ({num_classes}): {class_names}')
43 print(f'\nDataset curat: {df_clean.shape}')

```

```

Randuri eliminate (NaN/inf): 0
Randuri eliminate (duplicate): 0
Coloane eliminate (var=0): 1 -> ['num_outbound_cmds']...
Coloane categorice encodeate cu succes.
Eliminate 6 clase rare (<10 exemple)

Clase finale (17): ['back', 'buffer_overflow', 'guess_passwd', 'imap', 'ipsweep', 'land', 'neptune', 'rmap', 'normal', 'pod', 'portsweep', 'rootkit',
'satan', 'smurf', 'teardrop', 'warezclient', 'warezmaster']

Dataset curat: (125940, 42)

```

Figura 5: Rezultatele procesului de curatare a datelor (eliminarea duplicatelor si features inutile).

3.1 Feature Sampling

Pentru a evita situatia in care modelul invata sa recunoasca doar clasele majoritare (ex: traficul normal), aplicam tehnica de Undersampling, limitand numarul maxim de inregistrari la 50.000 per clasa.

```

1 from imblearn.under_sampling import RandomUnderSampler
2
3 # Features finale
4 feature_cols = [c for c in df_clean.select_dtypes(include=np
    .number).columns
5                 if c not in [label_col, 'label_enc']]
6
7 X = df_clean[feature_cols].values
8 y = df_clean['label_enc'].values
9
10 print(f'Features: {len(feature_cols)}')
11 print(f'Samples: {X.shape[0],}')
12
13 # Handling class imbalance cu RandomUnderSampler
14 max_per_class = 50_000
15 sampling_strategy = {}
16 for cls_idx, cls_name in enumerate(class_names):
17     count = np.sum(y == cls_idx)
18     sampling_strategy[cls_idx] = min(count, max_per_class)
19
20 rus = RandomUnderSampler(sampling_strategy=sampling_strategy
    , random_state=42)
21 X_res, y_res = rus.fit_resample(X, y)
22 print(f'\nDupa undersampling: {X_res.shape[0],} samples')
23
24 # Distributie dupa sampling
25 unique, counts = np.unique(y_res, return_counts=True)
26 for u, c in zip(unique, counts):

```

27

```
print(f' {class_names[u]}: {c:,}')'
```

```
Features: 40
Samples: 125,940

Dupa undersampling: 108,597 samples
back: 956
buffer_overflow: 30
guess_passwd: 53
icmp: 11
ipsweep: 3,599
land: 18
neptune: 41,214
nmap: 1,493
normal: 58,888
pod: 201
portsweep: 2,931
rootkit: 10
satan: 3,633
smurf: 2,646
teardrop: 892
warezclient: 890
warezmaster: 20
```

Figura 6: Noua distributie a claselor in urma procesului de Undersampling.

3.2 Training Splits 70-15-15

Impartim setul de date in trei submultimi: Antrenare (70%), Validare (15%) si Testare (15%). In plus, aplicam normalizarea pe attribute (StandardScaler) pentru a asigura stabilitatea algoritmilor de optimizare in antrenarea retelei neuronale.

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 # Split 70/15/15
5 X_train, X_temp, y_train, y_temp = train_test_split(
6     X_res, y_res, test_size=0.30, random_state=42, stratify=
7     y_res)
8 X_val, X_test, y_val, y_test = train_test_split(
9     X_temp, y_temp, test_size=0.50, random_state=42,
10    stratify=y_temp)
11
12 print(f'Train: {X_train.shape[0]:,}')
13 print(f'Val: {X_val.shape[0]:,}')
14 print(f'Test: {X_test.shape[0]:,}')
15
16 # Normalizare (fit doar pe train!)
17 scaler = StandardScaler()
18 X_train_s = scaler.fit_transform(X_train)
19 X_val_s = scaler.transform(X_val)
20 X_test_s = scaler.transform(X_test)
21 print('\nNormalizare aplicata.')
```

Train: 76,017
Val: 16,290
Test: 16,290

Figura 7: Dimensiunea subseturilor de antrenare, validare si testare.

4 MLP - Training

Începem dezvoltarea modelului Multi-Layer Perceptron (MLP) utilizând framework-ul PyTorch. Am definit o rețea neuronală feed-forward cu straturi ascunse, metode de regularizare (Dropout și BatchNorm) pentru a preveni fenomenul de overfitting, și un strat final de clasificare multiclasă.

```
1 import torch
2 import torch.nn as nn
3
4 device = torch.device('cuda' if torch.cuda.is_available()
5                       else 'cpu')
6
7 # Definire arhitectura MLP
8 class NIDS_MLP(nn.Module):
9     def __init__(self, input_dim, num_classes):
10         super().__init__()
11         self.net = nn.Sequential(
12             nn.Linear(input_dim, 256),
13             nn.BatchNorm1d(256),
14             nn.ReLU(),
15             nn.Dropout(0.3),
16
17             nn.Linear(256, 128),
18             nn.BatchNorm1d(128),
19             nn.ReLU(),
20             nn.Dropout(0.3),
21
22             nn.Linear(128, 64),
23             nn.ReLU(),
24             nn.Linear(64, num_classes)
25         )
26
27     def forward(self, x):
28         return self.net(x)
29
```

```

30 input_dim = X_train_s.shape[1]
31 mlp_model = NIDS_MLP(input_dim, num_classes).to(device)
32 print(mlp_model)
33 print(f'\nParametri totali: {sum(p.numel() for p in
    mlp_model.parameters()):,}')

```

```

NIDS_MLP(
  (net): Sequential(
    (0): Linear(in_features=40, out_features=256, bias=True)
    (1): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): Dropout(p=0.3, inplace=False)
    (4): Linear(in_features=256, out_features=128, bias=True)
    (5): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (6): ReLU()
    (7): Dropout(p=0.3, inplace=False)
    (8): Linear(in_features=128, out_features=64, bias=True)
    (9): ReLU()
    (10): Linear(in_features=64, out_features=17, bias=True)
  )
)
Parametri totali: 53,521

```

Figura 8: Arhitectura modelului MLP si numarul total de parametri.

4.1 Training Loop

Bucula de antrenare parcurge datele de antrenament timp de 30 de epoci. La sfarsitul fiecărei epoci, modelul este testat pe setul de validare, iar starea sa este salvata daca performanta se imbunatateste, pastrand astfel doar cea mai buna varianta.

```

1 import time
2
3 # Training loop MLP
4 EPOCHS = 30
5 history = {'train_loss': [], 'val_loss': [], 'train_acc':
    [], 'val_acc': []}
6 best_val_acc, best_state = 0.0, None
7
8 t0 = time.time()
9
10 for epoch in range(EPOCHS):
11     # --- Train ---
12     mlp_model.train()
13     t_loss, t_correct = 0.0, 0
14     for X_b, y_b in train_loader:
15         X_b, y_b = X_b.to(device), y_b.to(device)
16         optimizer.zero_grad()
17         out = mlp_model(X_b)
18         loss = criterion(out, y_b)
19         loss.backward()
20         optimizer.step()
21         t_loss += loss.item() * len(y_b)
22         t_correct += (out.argmax(1) == y_b).sum().item()
23
24     # --- Val ---
25     mlp_model.eval()

```

```

26     v_loss, v_correct = 0.0, 0
27     with torch.no_grad():
28         for X_b, y_b in val_loader:
29             X_b, y_b = X_b.to(device), y_b.to(device)
30             out = mlp_model(X_b)
31             v_loss += criterion(out, y_b).item() * len(
y_b)
32             v_correct += (out.argmax(1) == y_b).sum().item()
33
34     train_loss = t_loss / len(train_loader.dataset)
35     val_loss = v_loss / len(val_loader.dataset)
36     train_acc = t_correct / len(train_loader.dataset)
37     val_acc = v_correct / len(val_loader.dataset)
38
39     history['train_loss'].append(train_loss)
40     history['val_loss'].append(val_loss)
41     history['train_acc'].append(train_acc)
42     history['val_acc'].append(val_acc)
43
44     scheduler.step(val_loss)
45
46     if val_acc > best_val_acc:
47         best_val_acc = val_acc
48         best_state = {k: v.clone() for k, v in mlp_model.
state_dict().items()}
49
50     if (epoch + 1) % 5 == 0:
51         print(f'Epoca {epoch+1:02d}/{EPOCHS} | loss: {
train_loss:.4f} | val_loss: {val_loss:.4f} | acc: {
train_acc:.4f} | val_acc: {val_acc:.4f}')
52
53 mlp_model.load_state_dict(best_state)
54 mlp_train_time = time.time() - t0
55 print(f'\nAntrenare terminata in {mlp_train_time:.1f}s |
Best val_acc: {best_val_acc:.4f}')

```

```

Epoca 05/30 | loss: 0.0520 | val_loss: 0.0381 | acc: 0.9849 | val_acc: 0.9886
Epoca 10/30 | loss: 0.0340 | val_loss: 0.0262 | acc: 0.9895 | val_acc: 0.9924
Epoca 15/30 | loss: 0.0264 | val_loss: 0.0245 | acc: 0.9911 | val_acc: 0.9924
Epoca 20/30 | loss: 0.0245 | val_loss: 0.0220 | acc: 0.9925 | val_acc: 0.9936
Epoca 25/30 | loss: 0.0218 | val_loss: 0.0196 | acc: 0.9934 | val_acc: 0.9948
Epoca 30/30 | loss: 0.0199 | val_loss: 0.0185 | acc: 0.9936 | val_acc: 0.9947

Antrenare terminata in 41.0s | Best val_acc: 0.9948

```

Figura 9: Istoricul buclei de antrenare afisat in consola.

4.2 Curbe Antrenare MLP

Graficele de mai jos ilustreaza evolutia functiei de pierdere (Loss) si a acuratetii atat pe setul de antrenare cat si pe cel de validare, oferind o confirmare vizuala ca modelul converge corect, fara sa intre in overfitting masiv.

```
1 # Curbe antrenare MLP
```

```

2 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
3 epochs_range = range(1, EPOCHS + 1)
4
5 ax1.plot(epochs_range, history['train_loss'], label='Train')
6 ax1.plot(epochs_range, history['val_loss'], label='Val')
7 ax1.set_title('MLP - Loss')
8 ax1.set_xlabel('Epoca')
9 ax1.legend()
10
11 ax2.plot(epochs_range, history['train_acc'], label='Train')
12 ax2.plot(epochs_range, history['val_acc'], label='Val')
13 ax2.set_title('MLP - Accuracy')
14 ax2.set_xlabel('Epoca')
15 ax2.legend()
16
17 plt.tight_layout()
18 plt.show()

```

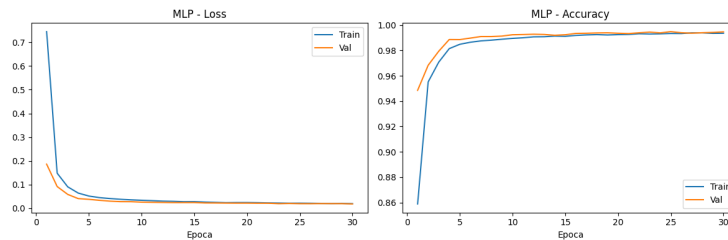


Figura 10: Graficele curbelor de antrenare (Loss si Acuratete) de-a lungul epocilor.

4.3 MLP Test-Set

Dupa ce modelul a fost antrenat, performanta lui finala este evaluata pe setul de testare (date pe care retea nu le-a vazut niciodata). Se afiseaza raportul de clasificare (Precision, Recall, F1-Score) si Matricea de Confuzie pentru a intelege exact intre care clase s-au facut greseli.

```

1 from sklearn.metrics import classification_report,
   confusion_matrix, f1_score
2 import seaborn as sns
3
4 # Evaluare MLP pe test set
5 mlp_model.eval()
6 all_preds, all_labels = [], []
7
8 with torch.no_grad():
9     for X_b, y_b in test_loader:
10         out = mlp_model(X_b.to(device))
11         all_preds.extend(out.argmax(1).cpu().numpy())

```

```

12         all_labels.extend(y_b.numpy())
13
14     y_pred_mlp = np.array(all_preds)
15     mlp_f1 = f1_score(all_labels, y_pred_mlp, average='weighted'
16                       )
17
18     print('=== MLP PyTorch - Classification Report ===')
19     print(classification_report(all_labels, y_pred_mlp,
20                               target_names=class_names))
21
22     # Confusion Matrix MLP
23     cm_mlp = confusion_matrix(all_labels, y_pred_mlp)
24     plt.figure(figsize=(15, 12))
25     sns.heatmap(cm_mlp, annot=True, fmt='d', cmap='Greens',
26               xticklabels=class_names, yticklabels=class_names
27               )
28     plt.title('MLP PyTorch - Confusion Matrix', fontsize=13)
29     plt.ylabel('Real')
30     plt.xlabel('Prezis')
31     plt.xticks(rotation=45, ha='right')
32     plt.tight_layout()
33     plt.show()

```

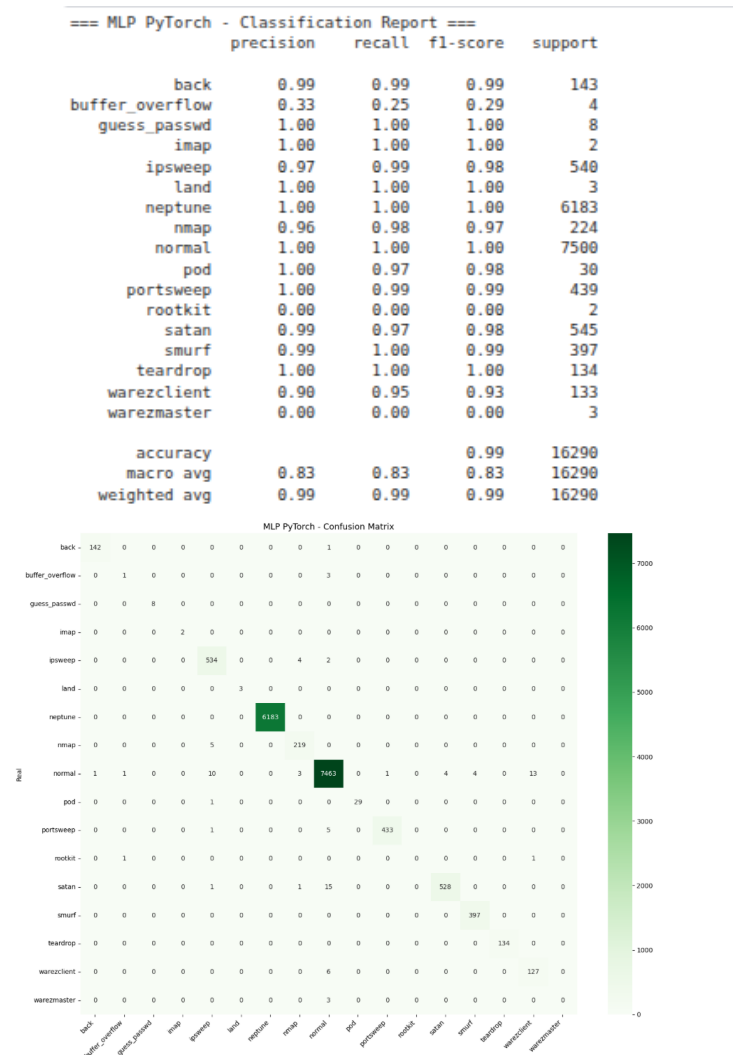


Figura 11: Rezultatele metrice pe setul de test (sus) si Matricea de Confuzie (jos).

5 Interfata

Pentru a transforma codul intr-un instrument utilizabil in practica, am dezvoltat o functie care simuleaza un sistem de inferenta in timp real. Acesta primeste date de trafic la cerere, le scaleaza si returneaza clasa prezisa, nivelul de incredere (confidence) si cele mai probabile 3 atacuri.

```

1 def predict_traffic(flow_features, model_type='xgb'):
2     """
3     Prezice tipul unui flux de retea.
4
5     Args:
6         flow_features: dict sau array cu valorile celor
7         features
8         model_type: 'xgb' sau 'mlp'
9
10    Returns:
11        (predicted_class, confidence)
12    """
13    if isinstance(flow_features, dict):
14        flow_arr = np.array([flow_features.get(f, 0) for f
15        in flow_features]).reshape(1, -1)
16    else:
17        flow_arr = np.array(flow_features).reshape(1, -1)
18
19    flow_scaled = scaler.transform(flow_arr)
20
21    if model_type == 'xgb':
22        pred = xgb_model.predict(flow_scaled)[0]
23        proba = xgb_model.predict_proba(flow_scaled)[0]
24    else:
25        mlp_model.eval()
26        with torch.no_grad():
27            t = torch.FloatTensor(flow_scaled).to(device)
28            logits = mlp_model(t)
29            proba = torch.softmax(logits, dim=1).cpu().numpy()
30
31        pred = proba.argmax()
32
33    predicted_class = class_names[pred]
34    confidence = proba[pred]
35    is_malicious = predicted_class.lower() != 'normal'
36
37    print(f'Clasa prezisa: {predicted_class}')
38    print(f'Confidence: {confidence*100:.1f}%')
39    print(f'Status: {"MALITIOS" if is_malicious else "NORMAL"}')
40    print()
41    print('Top 3 probabilitati:')
42    top3 = np.argsort(proba)[::-1][:3]
43    for i in top3:
44        print(f' {class_names[i]}: {proba[i]*100:.1f}%')
45
46    return predicted_class, confidence
47
48 # Test pe un exemplu din test set
49 print('== Inferenta pe primul sample din test set ==')
```

```

47 predict_traffic(X_test[0], model_type='mlp')
48 print(f'Label real: {class_names[y_test[0]]}')

```

```

=== Inferenta pe primul sample din test set ===
Clasa prezisa: neptune
Confidence:    100.0%
Status:        MALITIOS

Top 3 probabilitati:
neptune: 100.0%
normal:  0.0%
satan:   0.0%
Label real: neptune

```

Figura 12: Simulare de predictie folosind functia custom de inferenta.

6 Salvare Model MLP si XGBoost

Nu in ultimul rand, dupa ce obtinem modele performante, acestea impreuna cu obiectul StandardScaler trebuie salvate local. Astfel, ele pot fi integrate intr-un API sau intr-o aplicatie independenta fara a fi nevoie de o reantrenare.

```

1 import joblib
2
3 # Salvare StandardScaler si LabelEncoder pentru a le folosi
  la inferenta
4 joblib.dump(scaler, 'scaler_nslkdd.pkl')
5 joblib.dump(le, 'label_encoder.pkl')
6
7 # Salvare model PyTorch (MLP)
8 torch.save(mlp_model.state_dict(), 'mlp_nslkdd_model.pth')
9
10 # Salvare model XGBoost (presupunand ca instanta este numita
   xgb_model)
11 # xgb_model.save_model('xgboost_nslkdd_model.json')
12
13 print("Fisierele de model si scaler au fost salvate cu
   succes in directorul curent!")

```

7 Modificari aduse si diferente de grafice

In aceasta sectiune analizam vizual impactul modificarilor aduse arhitecturii si hiperparametrilor. Vom compara performanta modelului initial cu variante in care am eliminat regularizarea sau am modificat rata de invatare.

7.1 Teste comparative: Fara Dropout si LR marit

Primul bloc de cod ruleaza doua teste distincte. In Testul 1, eliminam straturile de Dropout pentru a forta modelul sa memoreze datele de antrenament (over-fitting). In Testul 2, pastram arhitectura originala, dar crestem masiv rata de invatare (Learning Rate la 0.01) pentru a demonstra cum optimizatorul devine instabil.

```
1 import time
2 import matplotlib.pyplot as plt
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from sklearn.metrics import accuracy_score
7
8 EPOCHS_TESTS = 15 # Folosim 15 epoci ca sa vedem rapid
   efectul
9
10 # TEST 1: ARHITECTURA ORIGINALA DAR FARA DROPOUT
11
12 print("INCEPE TESTUL 1: Fara Dropout")
13
14 class NIDS_MLP_NoDropout(nn.Module):
15     def __init__(self, input_dim, num_classes):
16         super().__init__()
17         self.net = nn.Sequential(
18             nn.Linear(input_dim, 256),
19             nn.BatchNorm1d(256),
20             nn.ReLU(),
21             # nn.Dropout(0.3) - ELIMINAT
22
23             nn.Linear(256, 128),
24             nn.BatchNorm1d(128),
25             nn.ReLU(),
26             # nn.Dropout(0.3) - ELIMINAT
27
28             nn.Linear(128, 64),
29             nn.ReLU(),
30
31             nn.Linear(64, num_classes)
32         )
33
34     def forward(self, x):
35         return self.net(x)
36
37 model_no_drop = NIDS_MLP_NoDropout(input_dim, num_classes).
   to(device)
38 opt_no_drop = optim.Adam(model_no_drop.parameters(), lr
   =0.001) # LR original
39 criterion = nn.CrossEntropyLoss()
```

```

40
41 hist_no_drop = {'train_loss': [], 'val_loss': [], 'train_acc': [], 'val_acc': []}
42
43 for epoch in range(EPOCHS_TESTS):
44     model_no_drop.train()
45     t_loss, t_correct = 0.0, 0
46     for X_b, y_b in train_loader:
47         X_b, y_b = X_b.to(device), y_b.to(device)
48         opt_no_drop.zero_grad()
49         out = model_no_drop(X_b)
50         loss = criterion(out, y_b)
51         loss.backward()
52         opt_no_drop.step()
53         t_loss += loss.item() * len(y_b)
54         t_correct += (out.argmax(1) == y_b).sum().item()
55
56     model_no_drop.eval()
57     v_loss, v_correct = 0.0, 0
58     with torch.no_grad():
59         for X_b, y_b in val_loader:
60             X_b, y_b = X_b.to(device), y_b.to(device)
61             out = model_no_drop(X_b)
62             v_loss += criterion(out, y_b).item() * len(y_b)
63             v_correct += (out.argmax(1) == y_b).sum().item()
64
65     hist_no_drop['train_loss'].append(t_loss / len(
66         train_loader.dataset))
67     hist_no_drop['val_loss'].append(v_loss / len(val_loader.
68         dataset))
69     hist_no_drop['train_acc'].append(t_correct / len(
70         train_loader.dataset))
71     hist_no_drop['val_acc'].append(v_correct / len(
72         val_loader.dataset))
73
74 # Evaluate Test 1
75 model_no_drop.eval()
76 preds_nd, labels_nd = [], []
77 with torch.no_grad():
78     for X_b, y_b in test_loader:
79         preds_nd.extend(model_no_drop(X_b.to(device)).argmax(
80             1).cpu().numpy())
81         labels_nd.extend(y_b.numpy())
82 acc_nd = accuracy_score(labels_nd, preds_nd)
83 print(f"-> Acuratete Test 1 (Fara Dropout): {acc_nd:.4f}\n")
84
85 # TEST 2: ARHITECTURA ORIGINALA (CU DROPOUT) DAR LR MAI MARE (0.01)

```

```

83
84 print("INCEPE TESTUL 2: Learning Rate crescut la 0.01")
85
86 class NIDS_MLP_HighLR(nn.Module):
87     def __init__(self, input_dim, num_classes):
88         super().__init__()
89         self.net = nn.Sequential(
90             nn.Linear(input_dim, 256),
91             nn.BatchNorm1d(256),
92             nn.ReLU(),
93             nn.Dropout(0.3), # READAUGAT (Original)
94
95             nn.Linear(256, 128),
96             nn.BatchNorm1d(128),
97             nn.ReLU(),
98             nn.Dropout(0.3), # READAUGAT (Original)
99
100            nn.Linear(128, 64),
101            nn.ReLU(),
102
103            nn.Linear(64, num_classes)
104        )
105
106        def forward(self, x):
107            return self.net(x)
108
109 model_high_lr = NIDS_MLP_HighLR(input_dim, num_classes).to(
110     device)
111 opt_high_lr = optim.Adam(model_high_lr.parameters(), lr
112     =0.01) # LR MULT MAI MARE
113 hist_high_lr = {'train_loss': [], 'val_loss': [], 'train_acc
114     ': [], 'val_acc': []}
115
116 for epoch in range(EPOCHS_TESTS):
117     model_high_lr.train()
118     t_loss, t_correct = 0.0, 0
119     for X_b, y_b in train_loader:
120         X_b, y_b = X_b.to(device), y_b.to(device)
121         opt_high_lr.zero_grad()
122         out = model_high_lr(X_b)
123         loss = criterion(out, y_b)
124         loss.backward()
125         opt_high_lr.step()
126         t_loss += loss.item() * len(y_b)
127         t_correct += (out.argmax(1) == y_b).sum().item()
128
129     model_high_lr.eval()
130     v_loss, v_correct = 0.0, 0
131     with torch.no_grad():
132         for X_b, y_b in val_loader:

```

```

130         X_b, y_b = X_b.to(device), y_b.to(device)
131         out = model_high_lr(X_b)
132         v_loss += criterion(out, y_b).item() * len(y_b)
133         v_correct += (out.argmax(1) == y_b).sum().item()
134
135         hist_high_lr['train_loss'].append(t_loss / len(
136             train_loader.dataset))
137         hist_high_lr['val_loss'].append(v_loss / len(val_loader.
138             dataset))
139         hist_high_lr['train_acc'].append(t_correct / len(
140             train_loader.dataset))
141         hist_high_lr['val_acc'].append(v_correct / len(
142             val_loader.dataset))
143
144     # Evaluate Test 2
145     model_high_lr.eval()
146     preds_hlr, labels_hlr = [], []
147     with torch.no_grad():
148         for X_b, y_b in test_loader:
149             preds_hlr.extend(model_high_lr(X_b.to(device)).
150                 argmax(1).cpu().numpy())
151             labels_hlr.extend(y_b.numpy())
152     acc_hlr = accuracy_score(labels_hlr, preds_hlr)
153     print(f"-> Acuratete Test 2 (High LR): {acc_hlr:.4f}\n")
154
155     # AFISARE GRAFICE COMPARATIVE
156
157     fig, axs = plt.subplots(2, 2, figsize=(14, 10))
158     ep = range(1, EPOCHS_TESTS + 1)
159
160     # Randul 1: Test 1 (Fara Dropout)
161     axs[0, 0].plot(ep, hist_no_drop['train_loss'], label='Train')
162     axs[0, 0].plot(ep, hist_no_drop['val_loss'], label='Val')
163     axs[0, 0].set_title('Test 1 (Fara Dropout) - Loss')
164     axs[0, 0].legend()
165
166     axs[0, 1].plot(ep, hist_no_drop['train_acc'], label='Train')
167     axs[0, 1].plot(ep, hist_no_drop['val_acc'], label='Val')
168     axs[0, 1].set_title('Test 1 (Fara Dropout) - Accuracy')
169     axs[0, 1].legend()
170
171     # Randul 2: Test 2 (High LR)
172     axs[1, 0].plot(ep, hist_high_lr['train_loss'], label='Train',
173         color='red')
174     axs[1, 0].plot(ep, hist_high_lr['val_loss'], label='Val',
175         color='orange')

```

```

171 axs[1, 0].set_title('Test 2 (High LR=0.01) - Loss (Observati
    instabilitatea)')
172 axs[1, 0].legend()
173
174 axs[1, 1].plot(ep, hist_high_lr['train_acc'], label='Train',
    color='red')
175 axs[1, 1].plot(ep, hist_high_lr['val_acc'], label='Val',
    color='orange')
176 axs[1, 1].set_title('Test 2 (High LR=0.01) - Accuracy')
177 axs[1, 1].legend()
178
179 plt.tight_layout()
180 plt.show()

```

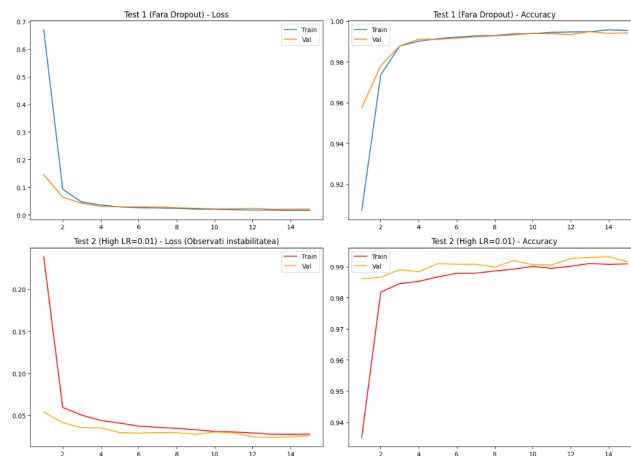


Figura 13: Grafice comparative pentru experimentele fara Dropout si cu rata de invatare ridicata.

7.2 Optimizarea Arhitecturii MLP (Varianta 2)

Pentru a testa influenta combinata a hiperparametrilor asupra performantei, am creat o a doua varianta de model. Modificarile fata de modelul initial sunt urmatoarele:

- **Mai putini neuroni / layere:** Am redus complexitatea (de la 256-128-64 la doar 64-32) pentru a vedea daca un model mai simplu se antreneaza mai repede fara sa piarda precizie.
- **Fara Dropout:** Am eliminat straturile de Dropout pentru a testa daca modelul tinde spre overfitting pe aceste date specifice.
- **Learning Rate modificat:** Am crescut rata de invatare de la 0.001 la 0.005 pentru a grabi convergenta intregii retele.

```

1 # EXPERIMENT: MLP Varianta 2 (fara Dropout, LR mai mare)
2
3 import time
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 from sklearn.metrics import accuracy_score, confusion_matrix
7 import torch
8 import torch.nn as nn
9 import torch.optim as optim
10
11 class NIDS_MLP_V2(nn.Module):
12     def __init__(self, input_dim, num_classes):
13         super().__init__()
14         self.net = nn.Sequential(
15             nn.Linear(input_dim, 64), # Mult mai putini
16             nn.BatchNorm1d(64),
17             nn.ReLU(),
18             # Am scos Dropout-ul complet
19
20             nn.Linear(64, 32),
21             nn.ReLU(),
22
23             nn.Linear(32, num_classes)
24         )
25
26     def forward(self, x):
27         return self.net(x)
28
29 # Initializam noul model
30 mlp_v2 = NIDS_MLP_V2(input_dim, num_classes).to(device)
31 print(mlp_v2)
32 print(f'\nParametri totali (V2): {sum(p.numel() for p in
33     mlp_v2.parameters()):,}')
34
35 # Folosim un Learning Rate mai mare (0.005 fata de 0.001)
36 optimizer_v2 = optim.Adam(mlp_v2.parameters(), lr=0.005)
37 criterion = nn.CrossEntropyLoss()
38
39 # Cream dictionarul pentru a salva istoricul graficelor
40 history_v2 = {'train_loss': [], 'val_loss': [], 'train_acc':
41     [], 'val_acc': []}
42
43 # Antrenam pe 15 epoci
44 EPOCHS_V2 = 15
45 print('\nAntrenare MLP Varianta 2...')
46 t0_v2 = time.time()
47
48 for epoch in range(EPOCHS_V2):
49     # --- Partea de Antrenare (Train) ---

```

```

48     mlp_v2.train()
49     t_loss, t_correct = 0.0, 0
50     for X_b, y_b in train_loader:
51         X_b, y_b = X_b.to(device), y_b.to(device)
52         optimizer_v2.zero_grad()
53         out = mlp_v2(X_b)
54         loss = criterion(out, y_b)
55         loss.backward()
56         optimizer_v2.step()
57
58         t_loss += loss.item() * len(y_b)
59         t_correct += (out.argmax(1) == y_b).sum().item()
60
61     # --- Partea de Validare (Val) ---
62     mlp_v2.eval()
63     v_loss, v_correct = 0.0, 0
64     with torch.no_grad():
65         for X_b, y_b in val_loader:
66             X_b, y_b = X_b.to(device), y_b.to(device)
67             out = mlp_v2(X_b)
68             v_loss += criterion(out, y_b).item() * len(y_b)
69             v_correct += (out.argmax(1) == y_b).sum().item()
70
71     # Calculam mediile pe epoca si le salvam in istoric
72     train_loss = t_loss / len(train_loader.dataset)
73     val_loss = v_loss / len(val_loader.dataset)
74     train_acc = t_correct / len(train_loader.dataset)
75     val_acc = v_correct / len(val_loader.dataset)
76
77     history_v2['train_loss'].append(train_loss)
78     history_v2['val_loss'].append(val_loss)
79     history_v2['train_acc'].append(train_acc)
80     history_v2['val_acc'].append(val_acc)
81
82     # Afisam progresul
83     print(f'Epoca {epoch+1:02d}/{EPOCHS_V2} | loss: {
      train_loss:.4f} | val_loss: {val_loss:.4f} | acc: {
      train_acc:.4f} | val_acc: {val_acc:.4f}')
84
85     mlp_v2_time = time.time() - t0_v2
86     print(f'\nAntrenare terminata in {mlp_v2_time:.1f}s')
87
88     # 1. AFISARE GRAFICE DE ANTRENARE
89
90     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
91     epochs_range = range(1, EPOCHS_V2 + 1)
92
93     ax1.plot(epochs_range, history_v2['train_loss'], label='
      Train')
94     ax1.plot(epochs_range, history_v2['val_loss'], label='Val'

```

```

    )
95 ax1.set_title('MLP Varianta 2 - Loss')
96 ax1.set_xlabel('Epoca')
97 ax1.legend()
98
99 ax2.plot(epochs_range, history_v2['train_acc'], label='Train
    ')
100 ax2.plot(epochs_range, history_v2['val_acc'], label='Val')
101 ax2.set_title('MLP Varianta 2 - Accuracy')
102 ax2.set_xlabel('Epoca')
103 ax2.legend()
104
105 plt.tight_layout()
106 plt.show()
107
108
109 # 2. EVALUARE RAPIDA PE SETUL DE TEST
110
111 mlp_v2.eval()
112 all_preds_v2, all_labels_v2 = [], []
113 with torch.no_grad():
114     for X_b, y_b in test_loader:
115         out = mlp_v2(X_b.to(device))
116         all_preds_v2.extend(out.argmax(1).cpu().numpy())
117         all_labels_v2.extend(y_b.numpy())
118
119 acc_v2 = accuracy_score(all_labels_v2, all_preds_v2)
120 print(f'\n=== Rezultat Experiment ===')
121 print(f'Acuratete MLP V2 pe datele noi (Test): {acc_v2:.4f}')
122
123 # 3. AFISARE MATRICE DE CONFUZIE PENTRU V2
124 cm_v2 = confusion_matrix(all_labels_v2, all_preds_v2)
125 plt.figure(figsize=(15, 12))
126 sns.heatmap(cm_v2, annot=True, fmt='d', cmap='Oranges',
127             xticklabels=class_names, yticklabels=class_names
128             )
129 plt.title('MLP Varianta 2 - Confusion Matrix', fontsize=13)
130 plt.ylabel('Real')
131 plt.xlabel('Prezis')
132 plt.xticks(rotation=45, ha='right')
133 plt.tight_layout()
134 plt.show()

```

Parametri totali (V2): 5,395

```

Epoca 02/15 | loss: 0.0811
Epoca 03/15 | loss: 0.0445

```

Antrenare terminata in 13.9:

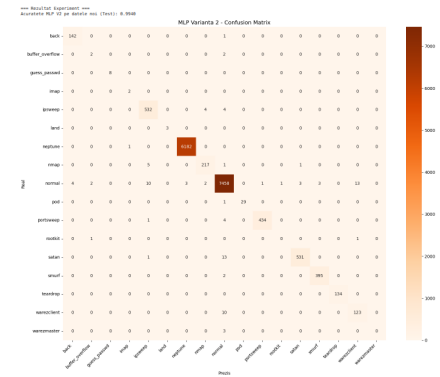
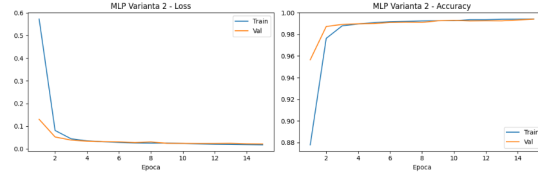


Figura 14: Curbele de antrenare si Matricea de Confuzie obtinute cu varianta 2 de arhitectura MLP.